CH 11

Concurrency & Locking

Optimistic · Pessimistic · Distributed · MVCC · CRDT

Overview

From JPA version columns to Redis distributed locks, ETags, MVCC, event sourcing, CRDT conflict resolution, and idempotency keys.

Locking Strategy Decision Tree

Deadlock Prevention

Rule: always acquire locks in ascending productId order.

```
// DeadlockPreventionService
productIds.stream().sorted().forEach(id -> acquireLock(id));
```

Idempotency & Event Sourcing

Component	Class	Mechanism
Idempotency Key	IdempotencyFilter	Idempotency-Key header → Redis lookup; return cached response if exists
Event Store	OrderEventStore (Cassandra)	Immutable append: ORDER_PLACED, PAYMENT_CAPTURED, ORDER_SHIPPE
Projection Rebuild	OrderProjection.rebuild(orderId)	Replay all events to reconstruct current Order state
Saga State	SagaStateStore (Redis)	Tracks PlaceOrderSaga state per orderId for correlation

MVCC & Snapshot Isolation

Scenario	Isolation Level	Class
Concurrent stock updates	REPEATABLE READ	OrderApplicationService.placeOrder()
Flash sale double-booking	SERIALIZABLE + @Retryable	WriteSkewPreventionService
Guest+customer cart merge	Last-write-wins (timestamp)	CartMergeService.merge()

HTTP Concurrency Controls

Header	Endpoint	Class	Behaviour
ETag (response)	GET /products/{id}	ShallowEtagHeaderFilter	Hash of product.version
If-None-Match (request)	GET /products/{id}	ProductController	304 if ETag matches
If-Match (request)	PATCH /orders/{id}	OrderController	412 Precondition Failed on version mismatch